50.012 Networks Project Report:
LLM Speedtest: Benchmarking Real-World LLM API Network Performance

Group 10
Albert Nguyen-Tran (1011204)
Daniel Truong (1011195)
Richard Zhang (1011179)
Jonathan Leung (1010869)

April 26, 2026

Abstract

As Large Language Models (LLMs) move into production, developers need more than model-quality scores: they need comparable measures of *operational* performance over real networks. Common leaderboards and academic benchmarks focus on accuracy and reasoning, not tokens per second (TPS), time to the first streamed model output (our metrics panel labels this **TTFB/TTFT**), or end-to-end completion latency. This report presents **AI Speedtest**, a Next.js web application that streams fixed prompts to multiple commercial LLM providers, measures those metrics in the browser and on the server, and crowdsources anonymized results into a shared Turso (libSQL) database for community-facing aggregates. We connect the work to the application layer (HTTPS and streaming), transport-level effects on perceived latency, and media-style streaming analysis of token throughput, and we document the engineering challenges of unifying heterogeneous provider APIs, Server-Sent Events (SSE) telemetry, and dual client-side versus server-proxied execution paths.

1 Introduction and Problem Statement

1.1 Domain, background, and motivation

Software delivery of intelligence has shifted: many products integrate LLMs through *vendor-hosted* HTTP APIs instead of on-premise weights. For those products, the "model" is inseparable from the *network and systems path* to it—DNS, TLS, routing to an edge, queueing, and the provider's autoscaling. Yet product teams still reason about user experience in terms of responsiveness (when the first words appear) and flow (steady token delivery), in addition to answer quality.

Benchmarking of "network experience" is familiar outside AI: end users routinely measure raw bandwidth and latency (e.g., Ookla Speedtest) or path diagnostics (ping, traceroute) to reason about last-mile and ISP behavior [1]. Enterprise acquisition activity in that space (e.g., Ookla's place in the measurement ecosystem) underlines that such tooling is not a curiosity but a reference layer for how applications behave on real networks. We argue that a parallel need exists for *application-layer* measurement tailored to *streaming* LLM APIs: a reproducible, comparable "speed test" for inference endpoints.

From a **50.012 Networks** perspective, an LLM chat call is a long-lived HTTP response that often uses chunked transfer and SSE-style event framing. TTFB captures connection establishment,

server scheduling, and time to first model output, while TPS and total latency reflect sustained throughput and completion time—echoing how one separates startup delay from bitrate in multimedia. Without vendor-neutral tooling, teams cannot rank providers and regions on *these* terms using the *same* prompt, token accounting rules, and placement (browser vs. server) assumptions.

1.2 Problem statement

We address the following problem: There is no widely adopted, cross-vendor, client-executable method for *systematically* measuring and sharing streaming LLM API performance (latency, throughput, cache-related behavior) *under* real-world path conditions, in a way that is (i) easy to run in a web browser, (ii) extensible to multiple providers, and (iii) amenable to crowdsourced aggregation for regional comparison.

The gap matters because: (1) *accuracy benchmarks* (e.g., MMLU, GSM8K) are orthogonal to operational scores [2, 3]; (2) vendor dashboards and tracing tools are often per-account or per-stack, not standardized for cross-provider A/B; (3) ad hoc one-off `curl` tests do not preserve metadata (location, run counts, or aggregate statistics) for a team or a class. Our project is to design, implement, and evaluate a concrete system that treats LLM API streaming as a *network-observable* service.

2 Literature Review

Existing research related to this project can be divided into three main areas: traditional network performance measurement, LLM capability benchmarking, and LLM inference-serving performance. Each area contributes an important part of the motivation for our project, but none fully addresses the problem of evaluating real-world LLM API responsiveness from the perspective of an application developer. Traditional network measurement research motivates the need for client-side benchmarking methodology. LLM capability benchmarks show how models are usually compared today, while also revealing that accuracy-based evaluation does not capture production usability. Finally, LLM serving research identifies latency and throughput as major systems-level concerns, which directly supports our choice of Time to First Token, tokens per second, and total latency as benchmark metrics.

2.1 Traditional Network Performance Measurement

Traditional speed tests and network diagnostic tools are designed to estimate the performance of an Internet connection using metrics such as latency, throughput, packet loss, and route behavior. MacMillan et al. compare Ookla Speedtest and Measurement Lab's NDT7, showing that even widely used speed tests can produce different results depending on test design, server infrastructure, and network conditions. This is important because it shows that "speed" is not a universal value; it depends on how the test is constructed, where the test server is located, and what part of the network path is being measured [4].

Han et al. make a similar point by analyzing the validity of Internet speed test tools and showing that reported performance can depend on factors such as server location, test duration, number of parallel connections, and congestion control behavior [5]. Sundaresan et al. also show that broadband measurements can vary depending on where the measurement is taken, since end-host behavior, gateway placement, and local network conditions can affect the observed result [6]. Together, these works suggest that network performance should be measured from the user-facing perspective, rather than assumed from provider claims or theoretical bandwidth values.

This directly informs the design of our project. Our AI Speed Test adopts the same general philosophy as traditional Internet speed tests: performance should be measured empirically from the client side using standardized test conditions. However, LLM APIs introduce a more complex measurement problem than ordinary broadband speed tests. A normal speed test measures how quickly data can be transferred between a client and a test server. In contrast, an LLM API call includes the network round trip, provider-side routing, possible queueing, model inference, and streamed token generation. Therefore, a slow LLM response cannot be explained by download speed or ping alone.

For this reason, our project adapts the speedtest concept to the application layer. Instead of measuring raw upload and download throughput, we measure LLM-specific performance indicators such as Time to First Token, tokens per second, and total latency. These metrics better reflect what a user actually experiences when interacting with an AI application. The traditional network measurement literature therefore supports our project's broader methodology, while also showing why a specialized benchmark is needed for LLM APIs.

2.2 Existing LLM Capability Benchmarks

Most existing LLM benchmarks focus on model capability rather than real-world API performance. Hendrycks et al. introduce MMLU, a benchmark designed to evaluate multitask language understanding across 57 subject areas, including mathematics, computer science, law, history, and other academic domains. MMLU is valuable because it provides a standardized way to compare model knowledge and reasoning ability across diverse tasks [7]. However, it does not evaluate whether a model is fast enough, stable enough, or cost-efficient enough for real-time use in production applications.

Liang et al. propose HELM, a broader framework for holistic language model evaluation. HELM expands model evaluation beyond accuracy by considering multiple dimensions, including calibration, robustness, fairness, bias, toxicity, and efficiency [8]. This is useful because it recognizes that model quality cannot be reduced to a single score. However, HELM is still primarily a model evaluation framework rather than a client-side API performance benchmark. It helps answer questions such as "Which model performs better on a task?" but not necessarily "Which provider gives the best real-time experience from my location and network conditions?"

This distinction is central to our project. Developers building LLM-powered applications often care about model quality, but they also care about responsiveness. For example, a customer support chatbot may need a low Time to First Token so that users feel the system is responding immediately. A code assistant may need high tokens per second so that long completions stream smoothly. A batch summarization tool may tolerate slower initial latency if the total completion time and cost are acceptable. These are deployment-specific concerns that are not captured by benchmarks focused mainly on reasoning accuracy.

Our project therefore complements existing LLM benchmarks rather than replacing them. MMLU and HELM help evaluate what a model can do, while our AI Speed Test evaluates how the model behaves as a real-world API service. This is especially important because many developers choose between providers not only based on intelligence, but also based on latency, throughput, reliability, region, and usability. By measuring these operational metrics, our benchmark targets the practical layer between model evaluation and production deployment.

2.3 LLM Inference-Serving Performance

Recent systems research on LLM serving shows that latency and throughput are major challenges in deploying generative models. Kwon et al. introduce vLLM and PagedAttention, focusing on efficient memory management for LLM serving. Their work shows that serving LLMs at scale requires careful management of key-value cache memory and batching, and that these systems-level choices directly affect throughput and latency [9].

Zhong et al. further separate LLM serving into two important phases: prefill and decoding. Their DistServe system is motivated by the observation that interactive LLM applications care about both Time to First Token and the time between output tokens. Time to First Token reflects how long the user waits before seeing the beginning of a response, while decoding performance determines how quickly the rest of the response streams after generation begins [10]. This directly supports our choice of benchmark metrics, since our project measures TTFB, tokens per second, and total latency rather than using a single generic "speed" value.

However, systems papers such as vLLM and DistServe generally study serving infrastructure under controlled or provider-side conditions. In contrast, application developers using commercial APIs such as OpenAI, Anthropic, Google, or Groq often do not have visibility into the provider's internal queueing, hardware, routing, or data center placement. From the developer's perspective, these providers behave as black-box network services. This means that empirical client-side measurement is still necessary: standardized prompts must be sent to each provider, and externally visible timing behavior must be recorded.

2.4 Research Gap and Project Positioning

The existing literature leaves a clear gap between network measurement, LLM capability evaluation, and LLM serving research. Traditional network speed tests measure Internet performance, but they do not capture LLM-specific behavior such as token streaming, Time to First Token, or inference delay. LLM capability benchmarks evaluate reasoning and knowledge, but they do not measure whether an API is responsive enough for production use. LLM serving research studies latency and throughput, but often from the provider or infrastructure side rather than from the perspective of an ordinary developer comparing black-box commercial APIs.

Our project addresses this gap by applying a speedtest-style methodology to LLM APIs. The AI Speed Test treats each provider as a networked application-layer service and measures standardized client-facing metrics: Time to First Token, tokens per second, total latency, provider, model, and geographic region. This design allows us to evaluate not only how intelligent a model is, but also how usable it is in real-world interactive settings.

The project also connects directly to networking concepts covered in the course. At the application layer, the benchmark relies on HTTP-based API communication and streamed responses. At the transport layer, observed latency is affected by round-trip time, connection setup, congestion, and the stability of the client-provider path. At the systems layer, the measured response includes provider-side queueing and model inference. By collecting results across providers, models, and regions, the tool creates a practical dataset that reflects how network and system factors combine to shape the user experience of LLM applications.

In this sense, our project does not simply ask which LLM is "best." Instead, it asks which LLM API performs best under real deployment conditions for a given user, location, and application need. This makes the project a bridge between traditional network performance measurement and modern AI infrastructure evaluation.

3 Approach, Justification, and Implementation

3.1 Our approach (summary)

We implemented **AI Speedtest** as a Next.js application (App Router, v16 in `package.json`) with:

- a React front end that runs benchmarks, displays live TPS and TTFB, and fetches **user location** (for geographic tagging) via a best-effort combination of IP geolocation and browser position (see `src/lib/geo.ts`);
- a **per-provider** server route, `/api/benchmark/[provider]`, that streams **SSE** events to the client while delegating to provider-specific stream iterators (`src/lib/providers/`);
- optional **browser-direct** API calls to providers that allow CORS (OpenAI, Google, Groq), and server-mediated streaming for Anthropic (no CORS), selected automatically in the `useBenchmark` hook;
- a **Turso/libSQL** SQLite store with `/api/results` to `INSERT` and query rows, plus aggregate statistics used by the Community Benchmarks table.

All server benchmarks use a single fixed `BENCHMARK_PROMPT` in `src/lib/constants.ts` so that runs are comparable in length and style.

3.2 Justification of design choices

Why a web app? It makes the tool accessible, demonstrates HTTP/TLS in the user agent, and allows both "my laptop to API" and "shared deployment server to API" perspectives when comparing against server-side provider SDKs in our route handlers.

Why Server-Sent Events? The benchmark route returns `Content-Type: text/event-stream` and `Cache-Control: no-cache` with a `ReadableStream` that enqueues `data: {JSON}\n\n` frames, mirroring how many LLM providers expose stream semantics and letting us reuse one UI parser for all providers in server mode (see `src/app/api/benchmark/[provider]/route.ts`).

Why both client- and server-side runs? Some APIs allow browser CORS, enabling a path that more closely matches "user in browser calls provider." Anthropic is restricted to the server path, isolating the API key in the deployment environment and avoiding CORS. This is an **intentional engineering trade-off**, not an oversight, reflected in `web/src/lib/client-providers/index.ts` (`supportsClientSide` map).

Why Turso/libSQL? A lightweight, serverless SQL database with file fallback (`file:local.db`) makes development reproducible in the classroom while supporting hosted Turso in production, enabling `GROUP BY` aggregates in `web/src/lib/db.ts` (function `getAggregateStats`) without operating a custom backend service.

3.3 System Architecture and Execution (Expanded)

AI Speedtest is implemented as a full-stack web measurement system with four layers: (i) browser UI and benchmark orchestration, (ii) API routes for benchmark execution and persistence, (iii) provider adapters for heterogeneous LLM streaming APIs, and (iv) a SQLite-compatible storage backend (local or Turso). This layered design preserves a uniform benchmark workflow while handling provider-specific protocol and payload differences.

At the frontend, the `SpeedTest` component coordinates provider selection, model selection, and run control, while `useBenchmark` acts as the runtime orchestrator. The hook maintains benchmark

state (running status, live metrics, aggregated session results, and error state), and routes execution through either a browser-direct path or a server-proxy path. This decision is made dynamically based on provider CORS compatibility and whether a user-supplied API key is present.

From a networking perspective, this dual-path architecture is important: browser-direct mode measures a path closer to end-user experience (client → provider), while proxy mode measures server-mediated behavior (client → app server → provider). The application labels these modes at runtime to avoid conflating measurements from different network paths.

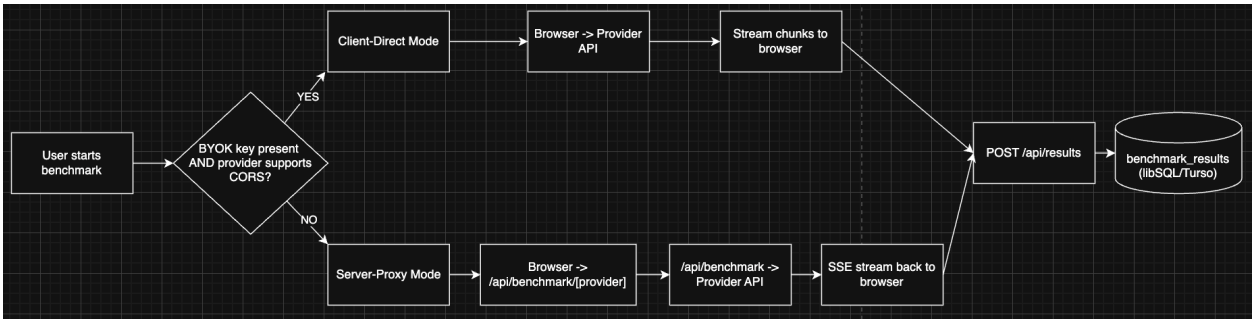

Figure 1: Dual execution path diagram: client-direct vs server-proxy and their measurement boundaries

3.4 Benchmark Execution Pipeline (Server-Side Flow)

When the client calls `POST /api/benchmark/[provider]`, the backend first validates the provider identifier against configured providers. It then resolves credentials (`body.apiKey` if provided, otherwise environment-based fallback) and resolves the model (request-selected model or default configured model). After validation, the route instantiates the appropriate provider adapter and starts an async streaming loop.

Provider adapters expose a unified async generator interface that emits standardized benchmark events: `start`, `chunk`, `complete`, and `error`. The API route serializes each event into Server-Sent Events (SSE) frames and streams them to the client (`Content-Type: text/event-stream`). This stream-first design lets the frontend compute and render metrics progressively without waiting for final completion.

On the client, streamed events are parsed incrementally and mapped into live metric updates. Buffering logic handles fragmented frames safely, because network chunk boundaries are not guaranteed to align with logical event boundaries. Malformed or partial records are ignored instead of terminating the run, improving robustness under real network conditions.

3.5 Metric Capture and Timing Semantics

The benchmark uses a monotonic timer (`performance.now()`) started immediately before provider invocation. Let t_0 denote request dispatch, t_1 the arrival of the first non-empty token chunk, and t_2 the final completion chunk.

- **TTFB** is measured as $t_1 - t_0$, representing startup delay before visible generation.
- **Total Latency** is measured as $t_2 - t_0$, representing end-to-end completion time.
- **Output Tokens** are accumulated across streamed chunks (provider usage metadata when available, otherwise estimator fallback).

- **Input Tokens** are captured from provider usage metadata when available.
- **TPS** is computed from output token count over elapsed benchmark duration according to implementation logic.

These measurements reflect application-layer user experience while remaining sensitive to transport and systems effects (connection setup, routing path, provider queueing, and decode speed).

3.6 Storage Integration and Data Path

On benchmark completion, the client submits a normalized result record to `POST /api/results`. The backend persists this record in `benchmark_results` via a libSQL client. The same query layer supports both local development (`file:local.db`) and remote shared deployment on Turso, ensuring consistent schema and query semantics across environments.

The storage schema captures three classes of information per run: identity (provider, model, timestamp), performance metrics (TTFB, TPS, total latency, token counts, cache status), and context (coarse location and server region). This enables both raw run inspection and aggregate analysis. Aggregate queries are computed server-side (e.g., mean TPS/TTFB and cache-hit ratio grouped by provider/model) and served directly to the community dashboard.

3.7 Engineering Challenges and Practical Trade-offs

To emphasize the implementation "do" aspect, we document the concrete failures and trade-offs encountered during development, and how they were resolved in code:

1. **SSE frame boundaries and stream reassembly.** Browsers may split HTTP response bodies arbitrarily, so an SSE event can arrive across multiple network reads. A naive parser that assumes one read equals one event causes JSON parse failures and dropped telemetry. In our server-consumption path, `useBenchmark` carries a persistent buffer and only parses complete `data:` records after splitting on `\n\n`. The browser-direct path applies the same defensive buffering strategy. Malformed records are skipped instead of aborting the benchmark, so one corrupted frame does not invalidate the full run.

2. **Heterogeneous provider stream formats.** Provider stream payloads are structurally different even when all are "chat streaming." OpenAI and Groq expose OpenAI-like deltas, while Gemini's `streamGenerateContent` nests token text and usage metadata differently. We centralized common SSE parsing and token-accumulation logic in shared helpers, while maintaining a Gemini-specific parsing loop where candidate text and usage fields require different extraction logic.

3. **CORS constraints vs. representative path measurement.** For network realism, browser-direct calls are preferred when possible because they measure the user-to-provider path directly. However, not all providers permit browser CORS access (notably Anthropic). We therefore implemented an explicit capability map in `client-providers/index.ts`: OpenAI/Google/Groq can run client-side, while Anthropic is forced through the server proxy. This avoids silent failures and makes measurement mode a deliberate, auditable choice.

4. **API key management across local and hosted modes.** The system needed to support both BYOK experimentation and hosted demonstrations. The benchmark route resolves

credentials via `body.apiKey||process.env[providerConfig.envKey]`. This unifies user-supplied keys with deployment-managed keys, and returns an explicit HTTP 400 when neither is present. The result is a predictable failure mode rather than ambiguous stream errors.

5. **Crowdsourced result integrity and queryability.** Raw crowd data can become hard to compare if aggregation is left to ad hoc client logic. We moved aggregation to the database layer: `insertResult` assigns generated unique IDs per record, indexes provider/country/time fields, and `GET /api/results?aggregate=true` computes per-(provider, model) summaries in SQL. This improves reproducibility and ensures that different UI views consume the same canonical aggregates.

6. **Attribution discipline (avoiding over-claims).** Commercial providers do not expose full internal routing or edge-placement metadata in a way we can reliably consume. To avoid false precision, we record *user-side* coarse geolocation and measured timings, but do not claim datacenter-level attribution. In particular, we treat TTFB as an end-to-end observable metric, not as a direct proxy for a known hop count or a single network segment.

4 Findings, Evidence, and Preliminary Results

In order to accurately capture the latency matrix, we separate data by Provider, Granular Model, and Geographic Region. This ensures that we are not comparing LLMs to each other, but rather how the network performance in a certain geographical location measures against model complexity.

Provider	**Model**	**Region**	**Avg TTFT (ms)**	**Avg TPS**	**Test Runs**
OpenAI	gpt-5.4	Singapore (SG)	1260	95.3	13
OpenAI	gpt-5.4-mini	Singapore (SG)	1607	217.1	10
OpenAI	gpt-5.4-nano	Singapore (SG)	800	208.8	10
Google Gemini	gemini-3.1-flash-preview	Singapore (SG)	15169	68.7	10
Google Gemini	gemini-3.1-flash-lite-preview	Singapore (SG)	792	239.9	10
Google Gemini	gemini-3.1-pro-preview	Singapore (SG)	13678	54.2	10
Groq	llama-3.3-70b-versatile	Singapore (SG)	479	531.6	11
Anthropic	claude-haiku-4-5	Singapore (SG)	910	107.6	11
Anthropic	claude-sonnet-4-6	Singapore (SG)	1146	62.1	10
Anthropic	claude-opus-4-7	Singapore (SG)	1125	76.9	10

Table 1: Per-provider performance in terms of TTFT (ms), TPS and number of test runs.

From the above table, the metrics of the 4 providers: Anthropic, Google Gemini, Groq, and OpenAI, was evaluated using the metrics: Average TTFT (Time To First Token), Average TPS (Tokens Per Second), and number of Test runs (Cache hit percentage was not recorded as every provider reported a cache hit of 0

Google Gemini reported exceedingly high TTFTs when it came to benchmarking "gemini-3-flash-preview" and "gemini-3.1-pro-preview" with TTFT values ranging from 4000ms- 40000ms and 12000ms– 15359ms. It can be argued that with the Gemini 3 flash preview, there may have been issues correlating with rate limits which causes a fluctuation in terms of reported TTFT; without the feature of crowdsourcing information, we would not be able to identify the issue with the fluctuating TTFT. Additionally, the Gemini 3.1 pro preview consistently showed a high TTFT of around 14000ms throughout all recorded tests, implying either a high deep thinking time or high server traffic within the provider's side.

Anthropic showed consistent TTFT, as well as average TPS across all models. The most noticeable characteristic of Anthropic's models is the consistent low TPS as compared to other models, even with slower performing models such as the Gemini 3 previews. However, it should be noted that powerful Claude models (such as Opus 4.7) perform on a similar metric as earlier models meaning that Anthropic may not struggle with traffic and token delivery as compared to Google Gemini across all models.

OpenAI is also notably consistent in terms of low TTFT and TPS across each model with a slightly higher TTFT and TPS than Anthropic across all models. It should be noted (although not shown), GPT 5.4 mini reported a TTFT of 9737ms for one of the records, inflating the TTFT value (possibly due to rate limiting); if we were to increase the number of runs across all models, we would see a lower TTFT for GPT 5.4 mini.

Groq showed a high TPS output, as well as the lowest recorded TTFT out of all the models. The efficient statistics can be attributed to Groq's hardware (rather than a network-related characteristic) as it uses a different infrastructure compared to the other 3 providers - Groq uses a Language which is built for sequential, low-batch workloads [16].

4.1 Sample Size and Interpretation Caution

The results in Table 1 should be interpreted as preliminary rather than definitive. Each provider-model pair currently has around 10–13 test runs, all collected from Singapore. This is enough to show that the benchmark pipeline works and to reveal early performance differences, but it is not enough to make universal claims about provider performance across all regions, times of day, network conditions, or traffic loads.

LLM API performance can vary due to temporary provider congestion, rate limits, routing changes, model load, cache behavior, and local network conditions. For example, a single unusually slow response can noticeably inflate the average TTFB when the number of runs is small. This is especially important for models where one or two test runs appear to be outliers. Therefore, the averages reported here should be treated as an initial snapshot of performance rather than stable long-term rankings.

A more rigorous evaluation would require a larger number of repeated trials, measurements across multiple geographic regions, and statistics beyond the mean, such as median, standard deviation, and 95th percentile latency. Despite these limitations, the preliminary results are still useful because they demonstrate that AI Speedtest can collect, store, and compare real streaming API measurements across providers using a shared methodology.

5 Future Work

Although the current version of the AI Speed Test establishes a working foundation for benchmarking LLM API performance, there are several important directions for future work. These extensions would make the benchmark more rigorous, more representative of real production conditions, and more useful for developers choosing between LLM providers.

5.1 Separating Network Delay from Inference Delay

One major limitation of the current benchmark is that the measured latency combines multiple sources of delay into a single observed value. When a request is sent to an LLM API, the total response time includes client-side overhead, DNS lookup, TCP/TLS connection setup, network round-trip time, provider-side routing, queueing, model inference, and streamed token generation.

From the user's perspective, these all contribute to the same delay, but from an engineering perspective, it is useful to understand which component is responsible for slow performance.

Future work could attempt to separate network delay from inference delay more carefully. For example, the system could collect baseline network measurements to provider endpoints, such as approximate HTTP request latency or round-trip time where available, and compare these values against the observed Time to First Token. If the baseline network latency is low but TTFB is high, this suggests that the bottleneck is likely provider-side queueing or model inference. If both baseline latency and TTFB increase together for a particular region, then the network path or provider routing may be a larger factor.

This would make the benchmark more diagnostic rather than only descriptive. Instead of simply reporting that one provider has a higher average TTFB, the tool could help developers reason about whether that delay is caused by network distance, routing behavior, server load, or model complexity. This extension would also strengthen the connection between the project and core networking concepts such as round-trip time, connection setup, and application-layer response delay.

5.2 Testing Prompt Size and Output Length

Another important area for future work is testing how prompt size and output length affect LLM API performance. The current benchmark can use standardized prompts to compare providers under similar conditions, but real applications vary significantly in the amount of input and output they require. For example, a chatbot may use short prompts and short responses, a code assistant may generate long outputs, and a summarization system may send long input documents but require shorter outputs.

Future versions of the benchmark could define multiple workload profiles based on input and output length. These could include short-prompt short-output tests, short-prompt long-output tests, long-prompt short-output tests, and long-prompt long-output tests. This would allow the tool to evaluate whether a provider performs consistently across different types of workloads. It would also help separate the effects of prompt processing from output generation. Longer prompts may increase the time required before the first token is generated, while longer outputs mainly affect total latency and tokens per second.

This extension is especially relevant because LLM serving performance depends on both the prefill stage and the decoding stage. By varying prompt length and expected output length, the benchmark could provide more detailed insight into where performance differences appear. A provider that performs well on short chatbot-style prompts may not perform as well on long-context summarization or code generation tasks. Adding workload-specific benchmark modes would therefore make the AI Speed Test more useful for developers with different deployment needs.

5.3 Expanding Geographic Testing

The current system records geographic information so that benchmark results can be grouped by region. Future work could expand this feature into a larger-scale geographic testing framework. Since LLM APIs are accessed through global network infrastructure, performance may vary depending on the user's physical location, the provider's edge routing strategy, and the distance to the serving region. A provider that performs well in one region may not necessarily provide the same responsiveness in another.

A more complete benchmark could collect results from users in multiple regions, such as Singapore, Canada, the United States, Europe, and other major deployment markets. These results

could then be visualized using regional latency maps, provider comparison charts, or location-based rankings. This would allow developers to answer questions such as which provider has the lowest TTFB in Southeast Asia, which model streams fastest from North America, or whether certain providers show more stable performance across regions.

Expanding geographic testing would also make the project more directly connected to networking concepts such as propagation delay, routing, CDN placement, and edge infrastructure. Since commercial LLM providers often do not expose the exact data center or edge node serving a request, client-side regional benchmarking provides a practical way to observe the effects of provider infrastructure from the outside. Over time, a crowdsourced dataset could reveal patterns in regional performance that are not visible from isolated local tests.

5.4 Adding Concurrency and Load Testing

The current benchmark focuses mainly on individual API requests. However, production applications often send many requests over a short period of time, especially when serving multiple users simultaneously. A provider that performs well for a single request may show degraded performance under higher load due to queueing, rate limits, or backend resource constraints. Therefore, future work should include concurrency and load testing.

A concurrency benchmark could send multiple simultaneous requests to the same provider and model while recording how TTFB, tokens per second, total latency, and error rate change as concurrency increases. For example, the system could compare performance at one, five, ten, or more parallel requests. This would help identify whether a provider maintains stable performance under load or whether latency increases sharply as request volume grows.

This extension would make the benchmark more representative of real deployment conditions. Developers building production systems need to know not only how fast a provider is under ideal single-request conditions, but also how it behaves during traffic spikes. Load testing could also reveal rate-limiting behavior, request queueing, or instability in streamed responses. By adding concurrency measurements, the AI Speed Test would move closer to a practical systems benchmark for LLM-powered applications.

5.5 Adding Reliability and Error-Rate Measurement

In addition to speed, future versions of the benchmark should measure reliability. A provider with low latency is not necessarily the best choice if it frequently fails, times out, or returns rate-limit errors. For production applications, reliability is just as important as raw performance because users expect consistent responses rather than occasional fast responses mixed with failures.

Future work could track request success rate, timeout rate, rate-limit frequency, provider-side errors, malformed streaming responses, and retry behavior. These metrics could be stored alongside TTFB, tokens per second, and total latency in the benchmark database. The results could then be used to compare not only which provider is fastest, but which provider is most stable across repeated trials and different regions.

This would make the benchmark more useful for real-world deployment decisions. For example, a slightly slower provider with a very high success rate may be preferable to a faster provider with frequent errors or inconsistent streaming behavior. Reliability metrics would also allow the benchmark to report performance distributions rather than only average values. By combining latency, throughput, and reliability, the AI Speed Test could provide a more complete picture of LLM API usability.

6 Conclusion

In this project, we motivated the need for vendor-neutral, streaming-oriented benchmarks for LLM APIs and implemented AI Speedtest as a concrete response to that gap. Existing LLM benchmarks mainly evaluate model capability, while traditional network tools measure raw connectivity and throughput. Our project bridges these two areas by treating LLM APIs as network-observable, application-layer services whose usability depends not only on model quality, but also on Time to First Token, tokens per second, total latency, routing behavior, provider infrastructure, and streaming stability.

The final implementation demonstrates this idea through a full-stack Next.js application backed by provider-specific streaming adapters, Server-Sent Events telemetry, dual client-direct and server-proxy execution paths, and a Turso/libSQL database for persistent benchmark aggregation. These implementation details were not only design choices, but also engineering challenges that had to be resolved in practice. In particular, handling fragmented SSE frames, normalizing heterogeneous provider response formats, managing CORS restrictions, supporting API-key flexibility, and preserving queryable crowdsourced results were necessary to make the benchmark usable across multiple commercial providers.

Our preliminary results show why this type of benchmark is useful. Across the Singapore test runs, providers and models showed clear differences in startup latency and streaming throughput. Groq produced the lowest average first-token delay and highest token throughput in our dataset, while several Gemini preview/pro models showed much higher TTFB values. Anthropic and OpenAI generally showed more consistent first-token behavior, though with different throughput profiles. These findings are not meant to declare a universal "best" LLM provider, since the dataset is still limited in size and geography. Instead, they demonstrate the value of measuring operational performance directly: models that appear similar from a capability perspective can feel very different when used as real-time networked services.

Overall, AI Speedtest provides a practical foundation for comparing LLM APIs using metrics that matter for deployment. It connects course concepts such as HTTP streaming, transport-layer latency, regional network variation, and application-layer throughput to a modern AI infrastructure problem. While future work should expand the number of regions, increase the number of trials, separate network delay from inference delay more carefully, and add reliability and load-testing metrics, the current system already shows that LLM API performance can be measured, stored, and compared in a standardized way. In this sense, the project moves beyond simply reading about LLM performance and instead builds a working measurement tool for evaluating how LLM APIs behave under real-world network conditions.

7 AI Usage Statement

AI tools were used throughout this project as a brainstorming, research, writing, and implementation support tool. We used AI to explore project directions, refine the problem framing, identify relevant research areas, and better understand technical concepts related to LLM benchmarking, streaming responses, latency, throughput, and network performance. AI was also used to review writing clarity, improve organization, and check alignment with the project rubric.

For implementation, AI was used as an iterative coding assistant. It helped us plan parts of the system architecture, reason through implementation options, debug errors, explain unfamiliar code behavior, and suggest changes for provider API integration, Server-Sent Events handling, metric calculation, CORS constraints, database storage, and frontend state management. Our workflow

involved implementing features, testing them, identifying issues, and then using AI feedback to iterate on the code and design.

Overall, AI assistance functioned like a technical tutor and pair-programming aid. It helped us understand trade-offs, troubleshoot implementation challenges, improve the clarity of the report, and refine the final system.

References

[1] Ookla, *How Speedtest Works* (methodology), `https://www.speedtest.net/insights/blog/how-speedtest-works/` (accessed 2026).

[2] D. Hendrycks et al., *Measuring Massive Multitask Language Understanding*, ICLR, 2021. (MMLU benchmark; widely used for static "reasoning/accuracy" leaderboards, orthogonal to our operational focus.)

[3] K. Cobbe et al., *Training Verifiers to Solve Math Word Problems*, arXiv:2110.14168 (GSM8K), 2021.

[4] K. MacMillan, T. Mangla, J. Saxon, N. P. Marwell, and N. Feamster, *A Comparative Analysis of Ookla Speedtest and Measurement Labs Network Diagnostic Test (NDT7)*, Proc. ACM Internet Measurement Conference, 2023. doi: 10.1145/3606376.3593522.

[5] J. Han, M. Nouwens, S. S. Kanhere, and V. Sivaraman, *On the Validity of Internet Speed Test Tools and Methodology for Measuring Broadband Access Network Performance*, Proc. Asian Internet Engineering Conference, 2022. doi: 10.1145/3570748.3570749.

[6] S. Sundaresan, W. de Donato, N. Feamster, R. Teixeira, S. Crawford, and A. Pescapè, *Broadband Internet Performance: A View From the Gateway*, Proc. ACM SIGCOMM, pp. 134–145, 2011. doi: 10.1145/2018436.2018452.

[7] D. Hendrycks et al., *Measuring Massive Multitask Language Understanding*, arXiv:2009.03300 (MMLU), 2020. doi: 10.48550/arXiv.2009.03300.

[8] P. Liang et al., *Holistic Evaluation of Language Models*, Annals of the New York Academy of Sciences, vol. 1525, no. 1, pp. 140–146, 2023. doi: 10.1111/nyas.15007.

[9] W. Kwon et al., *Efficient Memory Management for Large Language Model Serving with PagedAttention*, Proc. ACM SIGOPS 29th Symposium on Operating Systems Principles, pp. 611–626, 2023. doi: 10.1145/3600006.3613165.

[10] Y. Zhong et al., *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*, arXiv:2401.09670, 2024. doi: 10.48550/arXiv.2401.09670.

[11] LangChain, *LangSmith* documentation, `https://www.langchain.com/langsmith` (accessed 2026).

[12] Helicone, product documentation for LLM observability, `https://helicone.ai/` (accessed 2026).

[13] OpenAI, *API Reference: streaming responses (chat/completions)*—Streaming section, `https://platform.openai.com/docs/api-reference/streaming` (accessed 2026).

[14] Fielding, R. T. et al., RFCs on HTTP/1.1 and subsequent HTTP/2 work—background on request/response and multiplexing; relevant to discussing TLS+HTTP over modern LLM endpoints (general references from IETF).

[15] OpenTelemetry Project, `https://opentelemetry.io/` (accessed 2026).

[16] Inside NVIDIA Groq 3 LPX: The Low-Latency Inference Accelerator for the NVIDIA Vera Rubin Platform, https://developer.nvidia.com/ (accessed 2026).